PERL 5 ESSENTIALS

TABLE OF CONTENTS

1. INTRODUCTION & SETUP

About Perl

- **Practical Extraction and Report Language** - Text processing powerhouse
- Created by Larry Wall (1987), inspired by C, sed, awk, sh
- **Strengths**: Text manipulation, automation, rapid prototyping
- **Use Cases**: Web development (CGI), system admin, data processing

Windows Setup (Your Platform)

1. **Install Perl**:

```
winget install StrawberryPerl.StrawberryPerl
```

OR Download from strawberryperl.com

2. **Install Editor** (instead of Komodo):

```
winget install Vim.Vim
```

   Open with: `vim filename.pl`

3. **Verify Installation**:

```
perl --version
vim --version
```

Exercise Files: Create `C:\perl-exercises\` folder for all examples.

---

2. QUICK START

Hello World

```
#!/usr/bin/perl
# hello.pl
use strict;
use warnings;

print "Hello, World!\n";
```

Run: `perl hello.pl`

Counting Lines in File

```
#!/usr/bin/perl
use strict;
use warnings;

my $filename = "data.txt";
open my $fh, '<', $filename or die "Can't open $filename: $!";
my $lines = 0;
while (<$fh>) { $lines++; }
close $fh;
print "Lines: $lines\n";
```

Loops & Conditionals

```
#!/usr/bin/perl
use strict;
use warnings;
```

```
for my $i (1..5) {
    if ($i % 2 == 0) {
        print "$i is even\n";
    } else {
        print "$i is odd\n";
    }
}
```

Functions

```
#!/usr/bin/perl
use strict;
use warnings;

sub greet {
    my ($name) = @_;
    return "Hello, $name!";
}

print greet("World") . "\n";
```

Using perldoc

```
perldoc perl          # Perl overview
perldoc -f print      # Function help
perldoc -t perl       # Manual as text
```

---

3. BASIC SYNTAX

Anatomy of Perl Script

```
#!/usr/bin/perl          # Shebang (Windows: ignore)
use strict;              # Enforce variable declaration
use warnings;            # Show warnings

my $name = "Perl";       # Variable declaration
print "I love $name!\n"; # Statement
```

Component	Purpose	Example
`#!`	Shebang	`#!/usr/bin/perl`
`use strict`	Variable safety	Required!
`use warnings`	Error hints	Always use!

Component	Purpose	Example
Statements	Actions	`print "Hi";`

Statements & Expressions

- **Statement**: Does something (`print`, `my`)
- **Expression**: Produces value (`2+2`, `"hi"`)
- **Terminators**: `;` (except last statement)

Assignments

```
my $x = 42;           # Scalar
my @array = (1,2,3);  # List to array
my %hash = (a=>1);    # Hash
```

Whitespace & Comments

```
# Single line comment
print 1 +
       2;  # Whitespace ignored

=begin comment
Multi-line
comment block
=end comment
```

Blocks & Scope

```
my $x = 10;           # File scope

{
    my $x = 20;       # Block scope
    print "$x\n";     # 20
}
print "$x\n";         # 10
```

Chapter Quiz Answers: 11 questions covered in examples above.

---

4. VALUES & VARIABLES

Type	Sigil	Example	Description
Scalar	`$`	`$x = 42;`	Number/String
Array	`@`	`@x = (1,2);`	Ordered list

Type	Sigil	Example	Description
Hash	%	`%x = (a=>1);`	Key-value

Numeric Variables

```
my $pi = 3.14159;
my $int = 42;
print $pi + $int;  # 45.14159
```

Character Strings

```
my $single = 'single';
my $double = "double $int";  # Interpolation!
my $escape = "Line1\nLine2";
```

Quote Operators:

Operator	Example	Output
`q{}`	`q{Hello}`	`Hello`
`qq{}`	`qq{Hi $x}`	`Hi 42`
`qw{}`	`qw{a b c}`	`('a','b','c')`

Logical Values

```
my $true = 1;
my $false = 0;
my $truthy = "any string";  # True!
```

Lists & Arrays

```
my @fruits = qw{apple banana};  # List to array
print $fruits[0];               # apple (scalar context)
print @fruits;                  # apple banana
```

Slices:

```
my @subset = @fruits[0,2];  # First & third
```

Hashes

```
my %person = (
    name => "Alice",
    age  => 30
);
print $person{name};        # Alice
```

Constants

```
use constant PI => 3.14159;
print PI;  # 3.14159
```

---

5. CONDITIONALS

if Statement

```
my $age = 25;
if ($age >= 18) {
    print "Adult\n";
}
```

else/elsif

```
if ($age < 18) {
    print "Minor\n";
} elsif ($age >= 65) {
    print "Senior\n";
} else {
    print "Adult\n";
}
```

unless (Negative if)

```
unless ($age < 18) {  # Same as if ($age >= 18)
    print "Adult\n";
}
```

Switch (given/when)

```
use 5.010;  # Enable switch
my $fruit = "apple";
given ($fruit) {
    when ("apple") { say "Red" }
    when ("banana") { say "Yellow" }
    default { say "Unknown" }
}
```

Ternary Operator

```
my $status = $age >= 18 ? "Adult" : "Minor";
```

---

6. LOOPS

Loop Type	Use Case	Example
`while`	Condition first	`while ($i < 5)`
`until`	Opposite of while	`until ($i > 5)`
`for`	C-style	`for (my $i=0; $i<5; $i++)`
`foreach`	Iterate arrays	`foreach (@array)`

while/until

```
my $i = 0;
while ($i < 3) {
    print "$i\n";
    $i++;
}

until ($i > 5) { $i++; }
```

for Loop

```
for (my $i = 0; $i < 5; $i++) {
    print "$i\n";
}
```

foreach Loop

```
foreach my $fruit (@fruits) {
    print "$fruit\n";
}
# or shorthand:
for my $fruit (@fruits) { print "$fruit\n"; }
```

Loop Control

```
foreach (1..10) {
    last if $_ == 5;      # Exit loop
    next if $_ % 2 == 0;  # Skip even
    redo if $_ == 3;      # Restart iteration
}
```

---

7. SPECIAL VARIABLES

Variable	Sigil	Meaning	Example
`$_`	`$`	**Default scalar**	`print;` = `print $_;`
`@ARGV`	`@`	Command line args	`perl script.pl file.txt`
`` `$ ``	`` ` ``	`$`	Autoflush
`$!`	`$`	System error	`or die $!;`
`$.`	`$`	Line number	Current input line #
`$^W`	`$`	Warnings	`$^W = 1;`

Default Variable ($_)

```
while (<>) {  # Reads from files/STDIN
    chomp;        # Remove newline from $_
    print "Line $.: $_";
}
```

Function Arguments (@_)

```
sub sum {
    my $total = 0;
    $total += $_ for @_;  # All arguments
    return $total;
}
print sum(1,2,3);  # 6
```

---

8. OPERATORS

Category	Operators	Precedence
Arithmetic	`+ - * / % **`	High
Assignment	`= += -= *=`	Low
Relational	`== != < > <= >=`	Medium
Logical	`&& || !`	Low
File Test	`-e -f -d`	Medium

Arithmetic

```
my $x = 10 + 5 * 2;  # 20 (precedence!)
my $y = 10 ** 2;     # 100
```

Compound Assignment

```
$x += 5;  # $x = $x + 5
$x *= 2;  # $x = $x * 2
```

Relational & Logical

```
if ($x > 5 && $y < 10) { ... }
```

File Test Operators

```
if (-e "file.txt" && -f _) {  # _ = last file
    print "File exists and is a file\n";
}
```

Range Operator

```
for (1..5) { print; }  # 12345
```

String Concatenation

```
my $text = "Hello" . " " . "World";  # "Hello World"
```

---

9. REGULAR EXPRESSIONS

Pattern Match: =~ and !~

Basic Matching

```
my $text = "Hello World 123";
if ($text =~ /World/) { print "Found!\n"; }
```

Common Modifiers

Modifier	Meaning	Example
i	Case insensitive	/world/i
g	Global (all matches)	/o/g
s	Dot matches newline	/./s

Extracting Matches

```
if ($text =~ /(Hello) (\w+)/) {
    print $1;  # Hello
    print $2;  # World
}
```

Character Classes

Pattern	Matches
`.`	Any char
`\d`	Digit
`\w`	Word char
`[a-z]`	Letters
`[^0-9]`	Non-digit

Wildcards & Metacharacters

```
$text =~ /\d+/;      # One+ digits
$text =~ /^Hello/;   # Starts with
$text =~ /ld$/;      # Ends with
```

Search & Replace

```
$text =~ s/World/Perl/;  # Replace first
$text =~ s/World/Perl/g; # Replace all
```

Splitting

```
my @words = split /\s+/, $text;  # Split on whitespace
```

---

10. FUNCTIONS

Defining & Calling

```
sub name {          # Define
    return "Perl";
}
print name();       # Call
```

Arguments

```
sub greet {
    my ($name, $age) = @_;  # Unpack
```

```
    return "$name is $age";
}
```

Local Scope

```
sub example {
    local $x = 10;  # Temporary
}
```

Returning Values

```
sub multiply {
    return $_[0] * $_[1];
}
```

Static Variables

```
{
    my $count;
    sub counter { $count++; return $count; }
}
```

Predeclared Functions

- `print`, `say`, `length`, `chomp`, etc.

---

11. REFERENCES & STRUCTURES

Reference: Address of data (\ operator)

Array References

```
my $aref = [1,2,3];          # Anonymous array
my @array = (1,2,3);
my $aref2 = \@array;         # Reference to @array

print $aref->[0];            # 1
```

Hash References

```
my $href = { name => "Bob" };
print $href->{name};         # Bob
```

Mixed Structures

```
my $person = {
    name => "Alice",
```

```
    hobbies => ["reading", "coding"]
};
print $person->{hobbies}[0];  # reading
```

Reference Type

```
use Scalar::Util qw{reftype};
print reftype($aref);  # ARRAY
```

---

12. FILE I/O

File Handles

```
# Write
open my $fh, '>', 'output.txt' or die $!;
print $fh "Hello\n";
close $fh;

# Read
open my $fh, '<', 'output.txt' or die $!;
while (my $line = <$fh>) {
    chomp $line;
    print $line;
}
close $fh;
```

OO Interface

```
use IO::File;
my $fh = IO::File->new('>output.txt');
print $fh "Hello\n";
$fh->close;
```

Binary Files

```
open my $fh, '<:raw', 'image.jpg' or die $!;
```

Streams: <> reads from @ARGV or STDIN.

---

13. BUILT-IN FUNCTIONS

Category	Functions	Example
Output	print, say	say "Hi"; (auto-

Category	Functions	Example
		newline)
Error	`die`, `warn`	`die "Error!";`
String	`length`, `uc`, `lc`	`uc("hello");`
Numeric	`int`, `abs`, `rand`	`int(3.7);`
Array	`push`, `pop`, `shift`	`push @a, 4;`
Time	`time`, `localtime`	`scalar localtime;`

Key Examples

```
say "Hello";                    # Print + newline
die "Error at line $.\n";       # Fatal error
my $len = length("Perl");       # 4
push @fruits, "orange";         # Add to end
```

---

14. MODULES

Using Modules

```
use strict;
use Data::Dumper;           # External module
use Carp qw{croak};         # Core module

print Dumper(\%person);
```

Creating Module

MyModule.pm:

```
package MyModule;
use strict;

sub hello {
    return "Hello from module!";
}

1;  # True = success
```

Use it:

```
use lib '.';              # Local modules
use MyModule;
print MyModule::hello();
```

Carp for Errors

```
use Carp;
croak "Fatal error!";     # Better stack trace
```

---

15. BEST PRACTICES

Practice	Why	How
Always	`use strict; use warnings;`	Prevents bugs
Constants	Immutable values	`use constant MAX => 100;`
Comments	Explain WHY	`# Calculate tax rate`
Whitespace	Readability	2 spaces indent
Consistent	Teamwork	Always `my $var`

Example: Clean Code

```
#!/usr/bin/perl
use strict;
use warnings;
use constant PI => 3.14159;

sub circle_area {       # Clear name
    my ($radius) = @_;  # Explicit args
    return PI * $radius ** 2;
}

my $area = circle_area(5);
say "Area: $area";      # Clear output
```

□ NEXT STEPS CHECKLIST

Run ALL examples in `C:\perl-exercises\`

Complete quizzes (answers in examples)

Install: `cpanm` for modules: `winget install StrawberryPerl.cpanm`

Practice: Process a CSV file with regex

Project: Build a log file analyzer

Advanced: `perldoc perlreftut` (references)